РАЗРАБОТКА ИИ / БОЛЬШИЕ ЯЗЫКОВЫЕ МОДЕЛИ / БЕЗОПАСНОСТЬ

Более чистые данные для обучения ИИ, меньше ошибок: что такое SonarSweep от Sonar

SonarSweep фильтрует некачественные данные для обучения ИИ, сокращая количество ошибок и уязвимостей в сгенерированном коде на 41%. Узнайте, как это работает.

11 июня 2026 г. в 8:00 Джо Тайлер

Убайд Э. Аляфизи для Unsplash+

Компания Sonar спонсировала этот пост. Insight Partners инвестирует в Sonar и TNS.

Большие языковые модели быстро превратились из новинки в повседневную ›раструктуру для разработки программного обеспечения. Мы больше не ‹ользуем искусственный интеллект только для того, чтобы отвечать на

зачастую с такой скоростью, на которую не рассчитаны традиционные процессы проверки.

Доказано, что модели могут генерировать код. Вопрос в том, могут ли команды доверять тому, что они создают. Большая часть проблем возникает на более ранних этапах, в самих данных, используемых для обучения моделей — это яркий пример принципа «Мусор на входе — мусор на выходе». Исследовательская группа по искусственному интеллекту в компании Sonar тщательно изучила этот феномен и разработала технологию под названием SonarSweep, призванную решить эту проблему.

Публичные репозитории кода предоставляют моделям огромный выбор языков и фреймворков, но в них также содержатся устаревшие библиотеки, небезопасные шаблоны, хрупкие реализации и плохо поддерживаемые решения. Модели учатся на всем этом. Они не всегда могут отличить примеры, отражающие грамотную разработку, от примеров, которые просто компилируются.

Это важно, потому что функциональный код — это не то же самое, что готовый к использованию в продакшене код. В корпоративной среде код также должен быть безопасным, удобным для сопровождения и устойчивым к реальным условиям. Если эти качества непостоянны в обучающих данных, они будут непостоянны и в результатах.

Риск использования непроверенного кода

Большие языковые модели — это, по сути, сложные статистические системы. Они не обладают врожденным пониманием того, что такое «хорошая» и «плохая» разработка. Они оптимизируют наиболее вероятное решение на основе предоставленного контекста и закономерностей, выявленных на обучающих данных.

That means weaknesses in the training corpus can persist in model behavior in ways that are easy to underestimate. Research, including findings from Anthropic late last year, suggests that even relatively small samples of "poisoned" or low-quality data can be encoded into models' inner representations and lead to dangerous behaviors.

In a paper I co-authored last year, "Assessing the Quality and Security of AI-Generated Code: A Quantitative Analysis," we observed this exact issue. All models considered in our study generated a mixture of simple and sophisticated bugs, vulnerabilities, and code smells (maintainability issues), and there are some interesting trends between models: with certain labs prioritizing security performance while others succeed in writing maintainable code.

4. Tutorial: Building a Real-time Flight Tracker with Gemini Function Calling Capability

5. Small Language Models Are Driving a New Wave of Edge Native AI

To highlight the complexity of these issues, consider path traversal vulnerabilities. These typically require taint analysis (following an input source to a sensitive sink) in order to detect them. While an LLM might generate a block of code that performs a specific function correctly, it may fail to account for how unvalidated user input could manipulate file paths or inject malicious commands three functions down the line.

"A model can produce code that looks correct, passes a narrow test, and still introduces issues that increase review time, technical debt and security exposure."

For enterprise teams, this is the real risk. A model can produce code that looks correct, passes a narrow test, and still introduces issues that increase review time, technical debt and security exposure. At scale, those flaws do not stay isolated. They spread through pull requests, internal tools and downstream systems.

Why data quality engineering matters

Organizations are becoming increasingly interested in building and owning their AI. If they want their AI-assisted coding to be context-aware, trustworthy, and cost-effective, they need to treat their code and other data as critical assets. That means applying quality controls before the model ever learns from their datasets.

This is where data quality engineering becomes essential. Rather than accepting public code corpora (and even their own data) as-is, teams can inspect, filter, and improve training data so the model learns from stronger examples.

My team has put this into practice with SonarSweep, an approach that "sweeps" datasets before the model ever sees them. This ensures training data is properly ıtinized, which is crucial for companies seeking to understand and improve their

1. Analyze the code deeply. This goes beyond keyword filtering. Static analysis can identify bugs, security vulnerabilities, and maintainability issues across large datasets.
2. Synthesize examples. Create high-quality training examples for underrepresented tasks and domains. Optionally, an organization may use their codebase to embed specific understanding into the model.
3. Remediate what can be remediated. If insecure or outdated patterns can be automatically corrected, the model can learn from the improved version instead of the flawed one.
4. Curate aggressively. Not every example deserves equal weight in a training set. A quality gate removes low-signal data and maximizes diversity.

The payoff is measurable

This is not just a theoretical argument. Training on "swept" data in our model release from the end of last year led to a 41% reduction in the density of security vulnerabilities and a 41% reduction in the density of bugs in the model's generated output.

That kind of improvement matters because the ROI is not only technical. It is operational. Within an agentic coding session, when the model writes code containing fewer bugs and vulnerabilities, the agent will spend less time and fewer tokens in the loops of the Agent Centric Development Lifecycle (AC/DC) — a framework built for how software is developed today. Within the AC/DC method, AI agents generate most of the code, and teams manage the development loops in three core stages: Guide, Verify, and Solve.

Furthermore, training the model on your codebase will reduce token usage and help it get up to speed with your architecture and best practices at the start of a session. Additionally, cleaner, better-structured code reduces token use, as agents will spend less time re-reading files, rebuilding context, and working around unnecessary complexity.

Sonar's research supports that point: Across six matched pairs of codebases and roughly 660 Claude Code task runs, agents working in SonarQube-verified codebases used about 7% fewer input tokens and 8% fewer output tokens, with no meaningful change in task completion.

better foundations. We are reaching the point where the limiting factor is no longer how much code AI can produce. It is how quickly organizations can verify that code and decide whether it deserves to move forward.

> "The next frontier in AI coding is not simply bigger models or faster generation. It is better foundations."

To close the trust gap, organizations must build quality into their system from the beginning, including the datasets that shape model behavior before deployment. The teams that do this well will be the ones that get the most value from AI-assisted development, because their agents will be more efficient, spend less time correcting preventable defects, and have more time to put trustworthy software into production.

For engineer leaders and developers, the mandate is clear: development teams need to understand the implications of different agent configurations on their token spend and development output.

TNS

Joe Tyler is a specialist in Large Language Models with a background in natural language processing and real-world AI applications. He is currently an AI Researcher at Sonar based in London, leveraging generative AI to revolutionize code quality and security....

Read more from Joe Tyler →

TRENDING STORIES

Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development

4. Tutorial: Building a Real-time Flight Tracker with Gemini Function Calling Capability

5. Small Language Models Are Driving a New Wave of Edge Native AI

Receive a free roundup of the most recent TNS articles in your inbox each day.

svo1975pnv1982@gmail.com

SUBSCRIBE

Kubernetes
Observability
Operations
Platform Engineering

Newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS

Disclosures
Terms of Use
Advertising Terms & Conditions
Privacy Policy
Cookie Policy